

DON'T GO BROKE IN THE AGE OF AI

Before You Trust the First Draft

[COVER ILLUSTRATION — style-match required]

Where This Booklet Fits: This is booklet 4 of 5 in the Don't Go Broke series.

Disclaimer: This booklet provides general educational information and is not formal financial, legal, or professional engineering advice.

Table of Contents

- Field Note Opening
- Section 1 – Why the First Draft Feels More Trustworthy Than It Is
- Section 2 – The First Draft Review Session: A Walkthrough
- Section 3 – The "Looks Done vs. Is Done" Test
- Section 4 – The Screen and Data Review Protocol
- Section 5 – Review by User Role
- Section 6 – The Happy Path Trap
- Section 7 – What Did AI Actually Decide for You?
- Section 8 – Reviewing the Agent's Output: Files and Flows
- Section 9 – The Decision Guide: Accept, Revise, Revert, or Rebuild
- Section 10 – Before You Show It to Someone Else
- Field Note Closing
- Back Matter

Field Note Opening

You asked an AI builder to create the first version of your product. A few minutes later, it returned something that looks real.

There are screens. There are buttons. There is a working navigation menu. You might even have a working demo you can click through.

That creates a dangerous feeling: *"It looks real, so it must be right."*

The moment you receive the first AI-generated draft is the moment you are most vulnerable to false confidence. The project looks much closer to the finish line than it actually is. It is incredibly tempting to keep the momentum going, to add more features immediately, or to show the prototype to an investor or a potential customer.

Stop.

This is not the point where trust should begin. This is the point where rigorous review must begin. The blueprint you made before building is now the standard you must review the first draft against.

Notice how fast the excitement moves. Your mind is already drafting the message to your co-founder, the post announcing progress, the relief of finally being done. None of that is real yet. It is a feeling, not a fact, and feelings are exactly what a slick demo is built to produce.

The AI is not lying to you when it says the feature is built. To the AI, "built" means the code runs and the screen renders without an error — not that the data is real, the logic holds under pressure, or a stranger can't break it in thirty seconds. That judgment still has to come from you.

This is the moment where the discipline in this booklet pays off — while the draft is still new enough that a mistake is cheap to catch.

The first draft is not proof that the product is correct. It is simply the first thing that must be questioned.

Section 1

Why the First Draft Feels More Trustworthy Than It Is

When you build an app with AI, the results can be visually stunning within minutes. A polished interface, however, can hide weak thinking. A working demo can hide fragile logic. A confident AI response can hide a catastrophic architectural decision.

We are wired to believe that if something looks finished on the surface, the foundation must be solid. In traditional software development, reaching a working UI meant weeks of careful architectural planning, database design, and security modeling had already occurred.

With AI, the UI often comes first. The AI paints a beautiful exterior to mask a hollow or assumed foundation underneath.

This is not a flaw you can prompt your way out of. A model is optimizing for a response that looks complete and satisfies the literal request in front of it. It is not optimizing for the durability of your data model, the safety of your permissions, or whether your login flow survives real-world abuse. Those concerns only enter the process when you explicitly bring them in.

A five-minute build and a five-week build can look identical from the outside, because the visual layer takes about the same amount of time either way. The judgment about whether the decisions underneath are sound is the part AI cannot supply. That part is still entirely yours. Treat a beautiful first draft as a hypothesis, not a conclusion.

Momentum creates false confidence. Do not mistake a slick button for a sound application.

Section 2

The First Draft Review Session: A Walkthrough

Reviewing the first draft is not something you do passively while reading the AI's chat response. It is an active session. When the AI says, "I've built the first version," you must immediately switch from "Builder" mode to "Inspector" mode.

Here is the exact sequence of a safe review session:

1. **Pause Before Continuing:** Do not immediately ask the AI to add another feature. Do not click "Deploy." Do not send the link to a friend.
2. **Ask for the Change Receipt:** Force the AI to explicitly list what it built.
3. **Hunt for Assumptions:** Force the AI to confess what it guessed.
4. **Open the App as a User:** Step out of the codebase and click through the live demo.
5. **Test the Happy Path:** Verify that the core goal actually works.
6. **Break the Inputs:** Intentionally try to break the app.
7. **Test Alternate Roles:** Verify the app responds correctly to logged-out users, wrong users, and admins.
8. **Review the Files:** Look at the raw list of files the AI generated or modified.
9. **Decide on the Draft:** Make a final decision to Accept, Revise, Revert, or Rebuild.

The rest of this booklet provides the exact tools and worksheets you need to execute this session without needing an engineering degree.

A Walkthrough in Practice

Here is what this looks like with a real example. A founder asks an AI to build a simple internal tool: a way for a small team to log customer support tickets.

The AI responds in under ten minutes: "I've built the ticket logging system. You can create tickets, assign them, and mark them resolved."

Instead of opening the app immediately, the founder runs the sequence.

Pause Before Continuing: No new features requested yet. No deploy button clicked.

Ask for the Change Receipt: The AI lists four new files: a tickets table, a form component, a list view, and an update to the navigation menu.

Hunt for Assumptions: Asked directly, the AI admits it assumed every team member can see and edit every ticket, since no permission system was requested.

Open the App as a User: The founder clicks through. The form works. Tickets appear in the list.

Test the Happy Path: Creating a ticket, assigning it, and marking it resolved all work exactly as described.

Break the Inputs: The founder submits a ticket with no title. The app accepts it silently, creating a blank, unusable row in the list.

Test Alternate Roles: Logging in as a second, newly created user, the founder can see and edit every existing ticket — including ones assigned to other people.

Review the Files: Nothing unexpected. The four files match the four things that were asked for.

Decide on the Draft: This is a Revise, not an Accept. The core mechanic works, but two gaps need fixing before this ships: blank tickets should be rejected, and some kind of ownership or permission boundary needs to exist before a real team uses this.

Notice what did not happen here. The founder did not read a single line of code. They did not need to know what a database table is. They needed nine minutes and a willingness to click things the AI did not expect them to click. That is the entire skill this section is teaching — not technical fluency, just structured suspicion, applied in the same order every time.

[ILLUSTRATION — style-match required]

Section 3

The "Looks Done vs. Is Done" Test

If the app looks finished, you must actively prove that it isn't. The first step in reviewing a draft is learning to separate cosmetic completion from structural completion.

Use the **"Looks Done vs. Is Done" Test** to break the illusion of the first draft.

This distinction matters because "Looks Done" and "Is Done" answer two different questions. Looks Done asks whether the screen renders the way you pictured it. Is Done asks whether the thing behind the screen — the data, the validation, the security — actually behaves the way a real product needs to under real conditions, not just the conditions of a clean demo.

AI is exceptionally good at the first question and indifferent to the second, because nobody asked it to care. It will render a gorgeous dashboard full of numbers without ever being asked whether those numbers are real.

Run all three checks below before you accept any screen as finished, even if it looks polished enough to demo tomorrow.

The Test

1. The "Fake Data" Check

- **Looks Done:** The dashboard is full of beautiful charts and user profiles.
- **Is Done:** When you create a *new* account, the charts actually pull real data from your inputs.
- *Warning:* AI loves to invent hardcoded sample data (sometimes called "mock data") to make a screen look finished. If the data doesn't change when you interact with the app, the app is not done.

2. The "Form Validation" Check

- **Looks Done:** There is a beautiful "Contact Us" form with a glowing submit button.
- **Is Done:** If you try to submit the form without typing an email address, it stops you and shows a red error message.

- *Warning:* AI often builds forms that look great but don't validate bad inputs. If you can submit a blank form and get a "Success!" message, the form is dangerously incomplete.

3. The "Security Illusion" Check

- **Looks Done:** There is a login screen with an email and password field.
- **Is Done:** If you bypass the login screen and type the dashboard URL directly into your browser, the app kicks you out.
- *Warning:* An AI will happily build a login screen that is completely disconnected from the actual data. Just because a door exists doesn't mean the walls are locked.

Section 4

The Screen and Data Review Protocol

Once you establish that the app is structurally functioning, you must review exactly what it shows and how it behaves. Do not evaluate the app as a whole. Evaluate it screen by screen, and data point by data point.

Reviewing "the app" as a single unit is how serious problems hide. A dashboard, a settings page, and a checkout flow can each look equally finished while resting on completely different levels of real functionality underneath. One might be fully wired to the database and behaving correctly. Another might be quietly displaying data that never actually leaves your browser, or accepting input it never validates.

You cannot tell the difference by scrolling through the interface. Two screens can look like siblings and be built on entirely different foundations — one solid, one hollow. The only way to know which is which is to check each screen on its own terms, using the same questions every time, instead of trusting a general impression that the app "feels" finished.

The Screen Review Checklist

For every single generated screen, answer these questions:

- ☐ What is this screen supposed to do?
- ☐ What data does it pull from the database to show the user?
- ☐ What actions can the user take on this screen?
- ☐ What happens if the user views this screen but has no data yet?
- ☐ What happens if the user enters bad data or clicks the wrong button?
- ☐ What happens if the wrong user somehow accesses this screen?
- ☐ What data on this screen looks real but might be hardcoded fake data?
- ☐ What specific data point must be manually verified before I approve this screen?

The Data Behavior Review

A beautiful screen means nothing if the database behind it is broken. Use this beginner-safe guidance to review whether the draft actually saves, retrieves, protects, and updates data correctly.

- ☐ **The Persistence Check:** Does new data persist after a hard browser refresh?
- ☐ **The Privacy Check:** Can one user see another user's private data?
- ☐ **The Deletion Check:** When you click "Delete," does it actually delete the record from the database, or just hide it from the screen?
- ☐ **The Edit Check:** Does editing a record update the right one, or does it accidentally overwrite something else?
- ☐ **The Placeholder Check:** Are AI-generated placeholder records being mistaken for real user records?
- ☐ **The Exposure Check:** Is private information (like an API key, an email address, or a password) displayed anywhere in the code or on the screen where it shouldn't be?

[ILLUSTRATION — style-match required]

Section 5

Review by User Role

AI builders often suffer from tunnel vision. If you ask an AI to build a "User Dashboard," it will build a dashboard that works perfectly for the "Ideal User." It will forget that other types of users exist.

You must teach yourself to test the draft not just as the creator, but as different kinds of users interacting with the system under different conditions.

This tunnel vision is not a bug in the AI's reasoning — it is a direct result of how you prompted it. When you describe a feature, you are almost always describing it from your own point of view, or from the point of view of the ideal, cooperative user who does everything correctly and never gets confused.

The AI faithfully builds exactly that scenario and stops there. It will not spontaneously imagine the visitor who never logs in, the user who uploads a corrupted file, or the employee who tries to access something they were never given permission to see. Nobody asked it to think about them, so it didn't.

You have to imagine them for it, on purpose, every time — because if you don't, the first person to find that gap will be a real user, not you.

The Role Testing Matrix

Test the application acting as:

- **The Logged-Out Visitor:** Can they accidentally see the dashboard without logging in?
- **The New User:** What happens when they log in for the first time? Does the app crash because they have no data, or does it show a friendly onboarding screen?
- **The Returning User:** Does the app remember them correctly?
- **The Wrong User:** If Alice logs in, can she view Bob's profile by changing the URL?
- **The User with Messy Data:** What happens if a user uploads a 10MB image instead of a 1MB image?
- **The Admin/Owner:** Do they have special privileges to manage the platform? Are those privileges secure?

Case Study: The Admin Access Mistake

A builder asked an AI to create a marketplace where users could buy and sell digital templates. The builder also asked for an "Admin Dashboard" to see all transactions and ban misbehaving users.

The AI built it rapidly. The marketplace looked great. The Admin Dashboard looked powerful. The builder tested the Admin Dashboard and saw all the expected buttons.

What Looked Finished: The Admin Dashboard was fully functional. **What the AI Assumed:** The AI assumed that simply putting the Admin Dashboard on a hidden URL (`/admin-dashboard`) was enough security. It did not restrict access based on user accounts. **The Result:** A beginner could detect the issue by logging in as a regular user, typing `/admin-dashboard` into the URL bar, and discovering they had full power to ban other users.

What to Ask the AI: *"I noticed the admin dashboard is accessible by anyone who guesses the URL. Add strict role-based access control. Only users marked as 'Admin' in the database should be able to load this screen."*

In this case, the builder did not need to Rebuild the app. They just needed to Revise the security layer before continuing.

Section 6

The Happy Path Trap

The "Happy Path" is the sequence of clicks where the user does exactly what they are supposed to do, and the app responds exactly how it is supposed to respond.

A smooth demo usually only shows the Happy Path.

If you ask an AI to build a shopping cart, it will build a path where a user adds an item, clicks checkout, and sees a success screen. The AI will mark the feature as "complete" because this single path works perfectly.

But what happens when the user clicks the "Checkout" button twice rapidly? What happens if they empty the cart and then try to check out? What happens if their internet disconnects right after they pay?

AI skips edge cases because you did not explicitly ask for them. A working demo is often incredibly fragile beneath the surface. It looks fine when you follow the exact script the AI expects, but it breaks the moment you step off that path.

Happy Path vs. Real Use: A Worked Example

Imagine you asked an AI to build a simple signup flow for a newsletter.

What Looks Good at First (The Happy Path): You type your email, click "Subscribe," and a beautiful green checkmark appears. The AI declares the feature finished.

What the AI Assumed: The AI assumed that users will only ever click the button once, that they will always type a valid email address, and that the database will never fail to save the record.

What Breaks When Tested (Real Use): You decide to test the edges. You type "not-an-email" and click Subscribe. It shows the green checkmark anyway. You leave the field blank and click Subscribe. Green checkmark. You click the button rapidly five times. The database saves five identical blank records.

What You Should Ask Next: Instead of moving on to building the email-sending feature, you must stop and address the foundation. You do not need to rebuild the whole app, but you must instruct the AI to revise the logic: *"The UI looks good, but it accepts blank emails, invalid formats, and double-clicks. Add strict form validation and disable the button while loading."*

The Edge-Case Breaker Sheet

To review the first draft, you must intentionally step off the Happy Path.

- ☐ **The Double-Click Test:** Click every primary button (Submit, Save, Pay) twice very fast. Does the app crash? Does it charge the user twice?
- ☐ **The Empty State Test:** What happens when a user views a screen that has no data yet? (Does it show a helpful message, or does it crash trying to load an empty list?)
- ☐ **The Refresh Test:** Navigate deep into the app, then refresh your browser page. Does it remember where you are, or does it crash and send you back to the home screen?
- ☐ **The Back Button Test:** Fill out half a form, click the browser's "Back" button, and then click "Forward." Did it save your data, or did the app break?

[ILLUSTRATION — style-match required]

Section 7

What Did AI Actually Decide for You?

AI hates being stuck. If your prompt leaves out a detail, the AI will not stop and ask you for clarification. It will simply guess.

It will make assumptions about who can see what, how information is saved, what happens in unusual situations, and what the app does when you haven't told it what to do.

For example, you ask an AI to build a team dashboard. The AI might silently assume that anyone who creates an account should automatically have administrator access to view everyone else's financial data. It made a guess to keep the build moving.

You need to find these hidden decisions before you trust the code.

Hidden Assumption Case Study: The Fake Payment

A non-technical founder asked an AI agent to build a paid job board. They wrote a detailed prompt about the layout, the colors, and the pricing tiers.

The AI delivered a working prototype in ten minutes. The founder clicked "Post a Job," selected the \$50 tier, typed in a fake credit card, and the app showed a beautiful "Payment Successful! Your job is live!" screen. The job appeared on the homepage.

The founder was thrilled and immediately started showing the app to potential customers.

The Hidden Assumption: The AI had completely hallucinated the payment logic. Because the founder did not explicitly provide API keys for a real payment processor like Stripe, the AI wrote code that *simulated* a successful payment. It checked if a 16-digit number was entered, waited two seconds, and then just published the job to the database for free.

The founder could have detected this problem in two minutes without knowing how to read code, simply by interrogating the AI before trusting the draft.

The Hidden Assumption Finder

Use these questions to hunt down the AI's silent decisions:

- **Permissions:** Who is allowed to see this page? Did the AI assume everyone can see everything?
- **Data Persistence:** When I refresh the page, where does the data come from? Is it actually saved in a real database, or just temporarily stored in the browser's local memory?
- **Third-Party Services:** Did the AI sign me up for a paid service (like an email provider or a database host) in the code without asking me?
- **Simulated Behavior:** Is this feature actually working, or did the AI hardcode a "success" screen to make it look finished?
- **Scalability:** Did the AI write logic that works for 10 users but will crash if 1,000 users try to do it at the same time?

Section 8

Reviewing the Agent's Output: Files and Flows

Review does not require an engineering degree, but it does require a methodical approach. Do not start by looking at the colors or the fonts. Start with the files and the flows.

Colors and fonts are the easiest things to fix and least likely to hurt you. A file the AI quietly modified without being asked is the opposite — easy to miss and potentially expensive, especially if it touches authentication or data storage. Train your eye to look at structure before style: the file list is the most honest account of what actually changed, so read it before anything else.

File-by-File Review Rhythm

When an AI generates a draft, it outputs a list of files it created or modified. You must review this list critically using this rhythm:

1. **Look at Filenames First:** Do not read the code. Just read the names of the files the AI touched.
2. **Compare Names to the Task:** If you asked it to build a pricing page, you should expect to see files like `Pricing.jsx` or `pricing.css`.
3. **Flag Surprising Files:** If you asked for a pricing page, but the AI modified `UserAuthentication.ts` and `DatabaseConfig.js`, you have a massive red flag.
4. **Ask Why:** Ask the AI explicitly: *"Why did you modify the DatabaseConfig file when I only asked for a pricing page?"*
5. **Look for New Packages:** If the AI added a massive, complex third-party library just to format a date, challenge it.
6. **Avoid Approving the Unexplainable:** Do not approve changes to files if you cannot explain, in plain English, why the AI touched them.

A File List Worth Questioning

Imagine you asked an AI to add a "forgot password" link to your login page. Ten minutes later, it reports success and shows you the Change Receipt.

The file list includes: `LoginPage.jsx`, `ForgotPassword.jsx`, `EmailService.js`, `UserPermissions.ts`, and `package.json`.

Two of those five files should stop you immediately. `EmailService.js` might be reasonable — sending a reset email requires some way to send email — but you never discussed how. `UserPermissions.ts` has no obvious connection to a password reset link at all.

Asking why surfaces the answer fast. The AI explains that to send a secure reset link, it decided to add a temporary elevated permission so the reset process could bypass the normal login check. That is a real, working explanation — and also a meaningful new security surface that you did not ask for and would not have thought to test for, because you had no reason to expect it existed.

`package.json` changing is its own flag. A new dependency was added to handle the email sending. Is it a well-maintained, widely used package, or an obscure one the AI happened to pick? You do not need to be able to audit the code inside it, but you do need to know it is there, because every new dependency is one more piece of code you are now responsible for, whether or not you ever read a line of it.

None of this means the draft is wrong. It might be a perfectly reasonable way to solve the problem. But you cannot know that until you ask, and you cannot ask what you do not know exists. The file list is what tells you it exists.

[ILLUSTRATION — style-match required]

Section 9

The Decision Guide: Accept, Revise, Revert, or Rebuild

At the end of your review session, you must make a decision about what to do with the first draft. You do not have to keep everything the AI generated. You are allowed to throw parts of the draft away.

Use this decision guide to determine your next move:

1. Accept

When to use it: The draft does exactly what you asked. You used the "Looks Done vs. Is Done" test, and it passed. The AI surfaced its assumptions, and you agree with them. **Action:** Commit the code. Consider this feature a stable foundation. You may now move on to the next feature.

2. Revise

When to use it: The core logic is sound, but the AI missed a few edge cases. The empty states are missing, or the form doesn't validate inputs correctly. **Action:** Do not write a massive new prompt. Keep the current code, but issue narrow, specific instructions to fix the gaps. (e.g., "The login logic is good, but add error validation for missing email addresses.")

3. Revert

When to use it: You asked the AI to fix a small UI bug, but the Change Receipt shows it also rewrote your routing logic, updated three dependencies, and broke the navigation menu. **Action:** Throw the new changes away immediately. Use Git to revert to your last known good save point. Never ask the AI to "fix what it just broke." Restart the task with a stricter, narrower prompt.

4. Rebuild

When to use it: The draft is a mess of assumptions, hardcoded data, and over-engineered architecture. The UI looks nice, but the foundation is completely wrong for your actual goals. **Action:** Delete the entire draft. Do not try to patch a broken

foundation. It is infinitely faster to delete bad code and prompt the AI again with clearer, more specific instructions than it is to untangle a hallucinated architecture.

Before You Continue

Before you write the next prompt or add the next feature, you must definitively answer these questions about the draft you just reviewed. Do not proceed until you have clear answers.

- ☐ What is safe to keep?
- ☐ What is still fake?
- ☐ What is assumed?
- ☐ What is untested?
- ☐ What is risky?
- ☐ What must be rebuilt before adding features?
- ☐ What should be committed to version control right now?

Section 10

Before You Show It to Someone Else

There is a strong temptation to share the first draft immediately. You want feedback. You want validation. You want to show your investors or co-founders how fast you are moving.

But showing an unverified draft to collaborators or early users is incredibly dangerous.

If they see a polished UI, they will assume the app is almost done. They will start asking for tweaks to the button colors or the fonts rather than questioning the core logic.

Worse, if you show a demo that relies on hardcoded fake data or simulated payments, you are setting a false expectation of progress. This locks in bad assumptions and creates immense, artificial pressure on you to launch an app that isn't actually built yet.

Only share what you can explain. Sharing with others is incredibly useful, but only *after* you understand the core logic well enough to explain exactly what they are looking at and where the rough edges are. Make sure you understand what the draft is actually doing before handing it off.

The Cost of Showing Too Soon

A founder building a scheduling app for personal trainers had a working demo after one afternoon with an AI builder. Clients could book sessions, trainers could see their calendar, and everything rendered beautifully. Excited, the founder sent the link to three trainers that same evening, asking for feedback before doing any review of their own.

The trainers loved it. They started telling their clients to use it. Within a week, one trainer discovered that double-booking was possible — the app would happily schedule two clients into the same time slot, because nothing had ever checked for a conflict. Another trainer found that canceling a session did not free up the slot; the app just hid the canceled booking from the list while quietly keeping it reserved behind the scenes.

By the time these problems surfaced, real clients had been affected by them. The founder was not just fixing bugs anymore. They were apologizing to trainers, explaining to confused clients why their sessions had vanished, and trying to rebuild trust in a tool that people had already started to rely on.

None of this needed to happen. A single afternoon of the review process described in this booklet — testing the happy path, breaking the inputs, checking what happens when two people try to book the same slot — would have caught both problems before a single real client ever saw the app.

Sharing too early does not just risk an awkward conversation about an unfinished feature. It risks handing a live version of your product's mistakes directly to the people you can least afford to disappoint.

[ILLUSTRATION — style-match required]

Field Note Closing

Reviewing the first draft is not a delay from building; it is a critical part of building.

By taking the time to question the output, surface the assumptions, break the happy path, and verify the logic, you transform a fragile, generated draft into a trustworthy foundation.

You are not being paranoid. You are being professional. Every assumption you catch now is a problem you will not have to debug later, in front of a real user, at the worst possible time.

Build with your eyes open. Once the foundation is reviewed and secure, you can safely move on to iteration, refinement, and eventually, the responsibilities of a live launch.

That confidence is earned, not assumed — one interrogation, one tested edge case, one confirmed assumption at a time. Carry it forward.

The next booklet, *After Launch, the Meter Still Runs*, picks up exactly here. A reviewed draft does not stay still once it ships. It runs, and running costs money and carries risk you have not had to manage yet. You already know how to look closely instead of hoping for the best. The next booklet asks you to do the same thing for a live system instead of a first draft.

Back Matter

You have just spent nine sections learning how to look at a first draft with real scrutiny — hunting for hidden assumptions, testing the edges, checking every role, reading the file list instead of trusting the demo. That skill lives in your head now. This section turns it into paper.

The tools below are not optional extras. They are the difference between reviewing a draft once, under pressure, half-remembering the checklist from a booklet you read last week, and reviewing every draft the same rigorous way, every time, because the questions are sitting in front of you instead of buried in your memory.

Three tools live in this back matter, and each does a different job. The Interrogation Workflows give you exact prompts to copy and paste at the AI itself, forcing it to confess what it built and what it assumed instead of letting a vague "it's done" summary go unquestioned. The First Draft Testing Worksheet gives you a place to record what you actually tested, in real time, so nothing gets rounded up in your memory afterward. The Safe Approval Protocol is the final gate — a short, repeatable checklist you run before you let yourself move on to the next feature.

None of these tools require you to read code. They require you to ask the right question at the right moment, and to write down the answer instead of trusting you'll remember it. Use them in order: interrogate the AI first, test the draft yourself second, and only then run the final approval protocol before calling the feature done.

Skipping this step feels harmless in the moment. The draft looks good, you are in a hurry, and the checklist can wait until "later." But later is usually never, and the drafts that get waved through without this toolkit are exactly the ones that quietly break in front of a real user — the ones where the review would have caught the problem in five minutes instead of costing you an afternoon of firefighting after launch.

Consider what typically happens without a toolkit like this. A builder finishes reviewing a draft using nothing but a mental checklist. They remember to check the happy path. They forget to check what happens when a field is left blank, because that thought never surfaced without a physical reminder in front of them. The feature ships. Three weeks later, a real user leaves a field blank, the app breaks in a way nobody anticipated, and the builder is left debugging a problem that a single line on a worksheet would have caught before launch.

This is not a failure of intelligence or effort. It is a failure of memory under pressure, and pressure is the default condition of building something real. The tools in this back matter exist because relying on memory alone, every single time, for every single draft, is not a sustainable strategy — even for someone who is genuinely careful.

Treat these three tools as a single workflow, not three separate options to pick from. Run the interrogation prompts first, while the AI's explanation is still fresh and specific. Fill out the testing worksheet second, while you are actually clicking through the draft. Run the approval protocol last, as the final gate before you let yourself move on. Skipping straight to the approval protocol without the first two steps turns it into a rubber stamp instead of a real check, because you will be approving assumptions you never actually surfaced.

None of this requires becoming a programmer. Every prompt, every worksheet field, and every checklist item in this section is written in plain language on purpose. The skill this booklet teaches is not reading code — it is refusing to accept a confident-sounding summary as proof that something works. Bring that skepticism into every draft you review from here forward, and the tools below will do the rest.

The Interrogation Workflows

Do not passively accept the AI's summary of its work. Force it to explain itself using structured interrogation.

These are not "gotcha" questions designed to catch the AI in a lie. They are the same questions a careful technical cofounder would ask before signing off on someone else's work. You are just asking them yourself, in plain language, before you commit to keeping any of what was built.

The Change Receipt Interrogation

If the AI claims it "built the feature" or "fixed the bug," use this prompt to verify the true scope of the work:

"Output a detailed Change Receipt.

1. List every single file you modified or created.
2. For each file, explain in one plain-English sentence why it needed to be changed.
3. Did you modify any routing, authentication, styling, or dependency files? Answer 'Yes' or 'No' and explain.
4. Tell me what a human should manually verify before continuing."

The "What Did You Assume?" Interrogation

Copy and paste this into your chat to force the AI to confess its guesses:

"I am reviewing this draft.

1. List every technical, architectural, or user-experience assumption you made while generating this code that I did not explicitly request in my original prompt.
2. Specifically list assumptions related to data security, user permissions, edge cases, and hardcoded values.
3. List any fake, placeholder, or simulated behavior.
4. List any security, privacy, data, payment, or account risks introduced by these assumptions."

The First Draft Testing Worksheet

Use this worksheet to track your review session. Do not move on to the next feature until this worksheet is complete.

Fill it out in real time, while you are actually clicking through the draft — not from memory afterward. Memory tends to round up. It remembers the feature working and quietly forgets the one broken-input case you shrugged off because you were in a hurry. The worksheet does not forget.

Keep a completed worksheet for every feature you accept. Six months from now, when something breaks in production, this is the record that tells you what was actually tested and what was simply assumed to be fine.

Here is what a completed line might look like in practice, for a login feature:

Task Being Reviewed: Login and password reset flow. *What the AI claims is done:* "Login is complete, including email-based password reset." *Happy Path Result:* Pass — logging in with correct credentials works. *Broken-Input Result:* Crashed — entering an email with no @ symbol threw a raw error message instead of a friendly one. *Empty-State Result:* Handled — a new user with no account correctly sees a "create an account" prompt. *Refresh Result:* Handled — refreshing mid-login keeps the user on the login screen instead of losing their place. *Security Result:* Yes — a second test account was able to view the first account's profile page by changing the URL. This must be fixed before anything else. *Assumptions Found:* AI assumed anyone with a valid-looking reset link should be let in, without checking whether the link had expired. *Files Changed Unexpectedly:* A permissions configuration file was modified without being asked. *Final Decision:* Revise.

Notice that this single filled-out line already surfaced a real security problem, a validation gap, and an unexplained file change — three things a quick glance at the login screen would never have revealed. That is the entire value of writing it down instead of trusting your impression of how the demo felt.

Use one worksheet per feature, not one per app. A "login feature" and a "checkout feature" deserve separate entries, because passing one tells you nothing about whether the other is safe.

Task Being Reviewed: _____

☐ **What the AI claims is done:** (e.g., "Login flow is complete.")

- ☐ **Happy Path Result:** Did it work when used perfectly? [Pass/Fail]
- ☐ **Broken-Input Result:** What happened when you entered bad data?
[Handled/Crashed]
- ☐ **Empty-State Result:** What happened when there was no data?
[Handled/Crashed]
- ☐ **Refresh Result:** What happened when you refreshed the page mid-flow?
[Handled/Crashed]
- ☐ **Security Result:** Can the wrong user see private data? [Yes/No]
- ☐ **Assumptions Found:** _____
- ☐ **Files Changed Unexpectedly:** _____

Final Decision (Circle One): Accept / Revise / Revert / Rebuild

The Safe Approval Protocol

Before you declare the first draft complete and move on to building the next major feature, execute this final protocol.

This protocol is short on purpose. It is meant to take five minutes, not fifty, and it is meant to be repeatable every single time you accept a draft — not just the first time, when everything feels new and you are being unusually careful.

The risk is never highest at the start. It is highest a few features later, once reviewing has started to feel like a formality instead of a habit, and you begin waving drafts through because the last three happened to be fine. Run through every line below before you move on, even when the draft looks obviously ready. Especially when it looks obviously ready.

Two items on this list are easy to rush past. "Test the Roles" does not mean glancing at the app while logged in as yourself — it means actually creating a second account, or logging out entirely, and confirming the app behaves correctly from that different vantage point. It is the single most commonly skipped step, because testing as yourself always looks fine.

"Explain it Simply" is not a formality either. If you cannot describe, in one or two plain sentences, how the login check works or why the payment step is safe, that is a signal you are approving something you do not actually understand — and something you do not understand is something you cannot debug later when it breaks.

Running this protocol takes less time than writing the prompt that generated the draft in the first place. That asymmetry is the whole argument for doing it: a few minutes of structured checking now is cheap insurance against hours of confused debugging later, once the feature is buried under three more features built on top of it.

- ☐ **Review the Receipt:** I have read the Change Receipt and understand why every modified file was touched.
- ☐ **Test the Flow:** I have stepped off the Happy Path and tested broken inputs and empty states.
- ☐ **Test the Roles:** I have viewed the app as a logged-out visitor, a new user, and an admin.
- ☐ **Verify the Reality:** I have confirmed there is no fake, simulated, or placeholder behavior hiding behind the UI.
- ☐ **Approve the Assumptions:** I have interrogated the AI for its hidden guesses regarding permissions and data, and I approve them.
- ☐ **Save the Notes:** I have saved the results of my Testing Worksheet.
- ☐ **Explain it Simply:** If someone asked how this core logic works, I could explain it to them in plain English.

If you can check every box, the draft is "reviewed enough to continue."

[NEXT BOOKLET COVER — insert when available]